PyTorch

Fashion-MNIST

LECTURE 8 | GÉRON CH. 10

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 7 fitted a network with `MLPClassifier` and found four things it will not let you do:

you want to	you cannot
change what it optimises	there is no argument for it
see a gradient	nothing is exposed
stop mid-epoch, or log per batch	the smallest unit is one pass
move the computation to another device	there is no device to move to

> **ALL FOUR ARE ONE WALL**
>
> `fit()` contains a loop over epochs and batches that you did not write, cannot see and cannot enter. The repair is not a better library. It is owning the loop.

Today: what that loop has to compute, why it can be computed at all, and how to write it.

Today

1. **The mathematics** — backpropagation as reverse-mode automatic differentiation
2. What the wrapper costs, and when an accelerator pays
3. **The method** — tensors, autograd, and the training loop written out in full
4. Three bugs that never raise an exception, measured
5. **Design exercise** — the training pipeline behind a ride-hailing app's arrival-time model, in nine steps, with the pitfalls and how to address them

Open Lecture 8 in Google Colab

The question

You called `clf.fit()`.

It computed 266,610 partial derivatives, on every batch, 20 times over. *How?*

> **WHY YOU NEED THE ANSWER**
>
> Because the cost of that computation is what decides whether training a network is possible at all — and the reason it is possible is a property of the *direction* in which the chain rule is applied.

The chain rule, as a product of Jacobians

A network is a composition. Write the loss as

$$L = f_3(f_2(f_1(\boldsymbol{\theta}))) \qquad \Longrightarrow \qquad \frac{\partial L}{\partial \boldsymbol{\theta}} = J_3\, J_2\, J_1$$

where J_i is the Jacobian of f_i evaluated at the point the forward pass reached. Every layer contributes one factor.

✓ Matrix multiplication is associative.

So the product can be bracketed from either end. $(J_3 J_2)\, J_1$ starts at the output — **reverse mode**. $J_3\, (J_2 J_1)$ starts at the parameters — **forward mode**. Same number; very different cost.

Reverse mode · propagate an adjoint

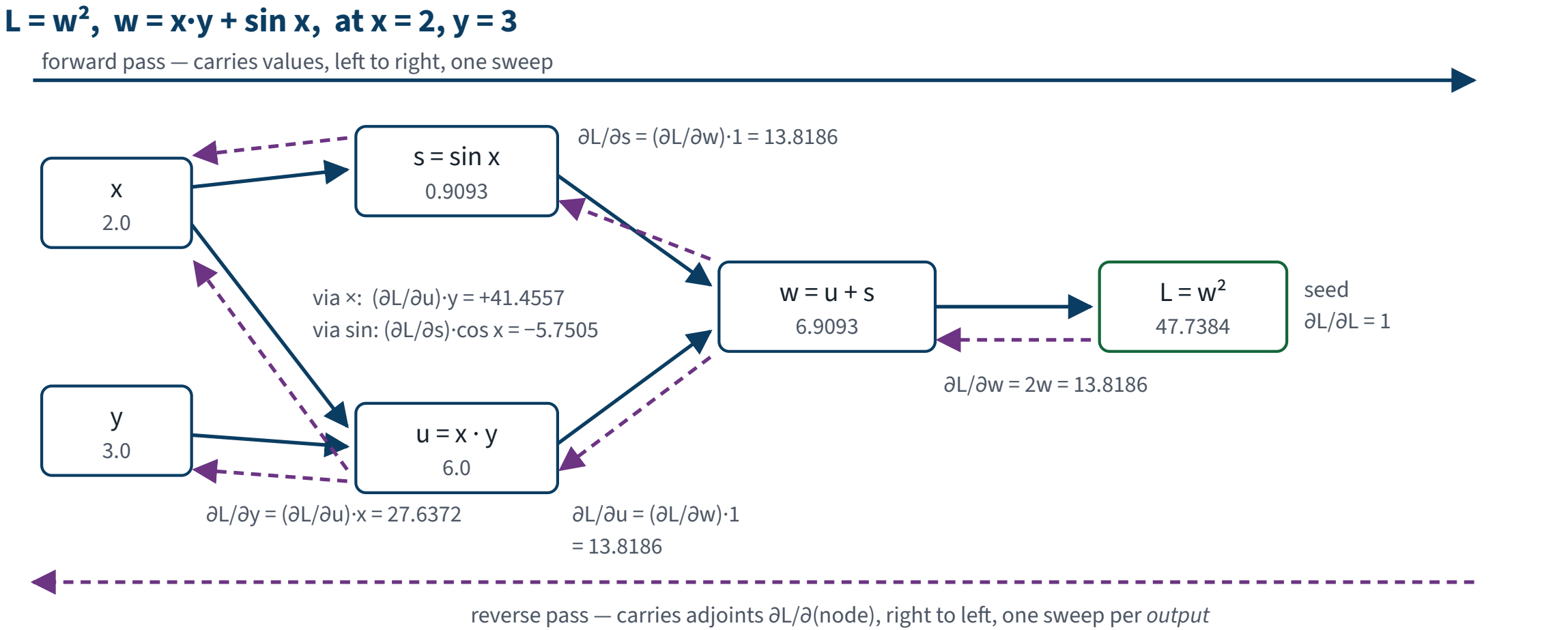
Pick a direction in *output* space — for a scalar loss there is only one, and it is 1. Carry $\bar{u} = \partial L / \partial u$ backwards:

$$\bar{u}_{\text{in}} \;=\; \bar{u}_{\text{out}} \, \frac{\partial f}{\partial u_{\text{in}}}$$

- One sweep gives the derivative of **one output** with respect to **every input**
- The forward values must be kept, because the Jacobians are evaluated at them — that is the memory cost

Forward mode, bracketed from the other end, gets one *input* direction per sweep, so a gradient over n parameters costs n sweeps. Reverse mode costs one sweep per *output* — and a loss has exactly one.

The whole derivation, in one picture



One sweep forward fills in the values, one sweep back every derivative — and x, used twice, **sums** its two edges.

So: what is backpropagation?

> **IN ONE SENTENCE**
>
> Reverse-mode automatic differentiation, applied to a neural network, with the seed set to 1 at the loss.

- It is not specific to neural networks
- It is not an approximation
- It is not gradient descent — it *supplies* the gradient that descent then uses

The 1986 paper's contribution was noticing that this was cheap enough to be practical, not inventing the chain rule.

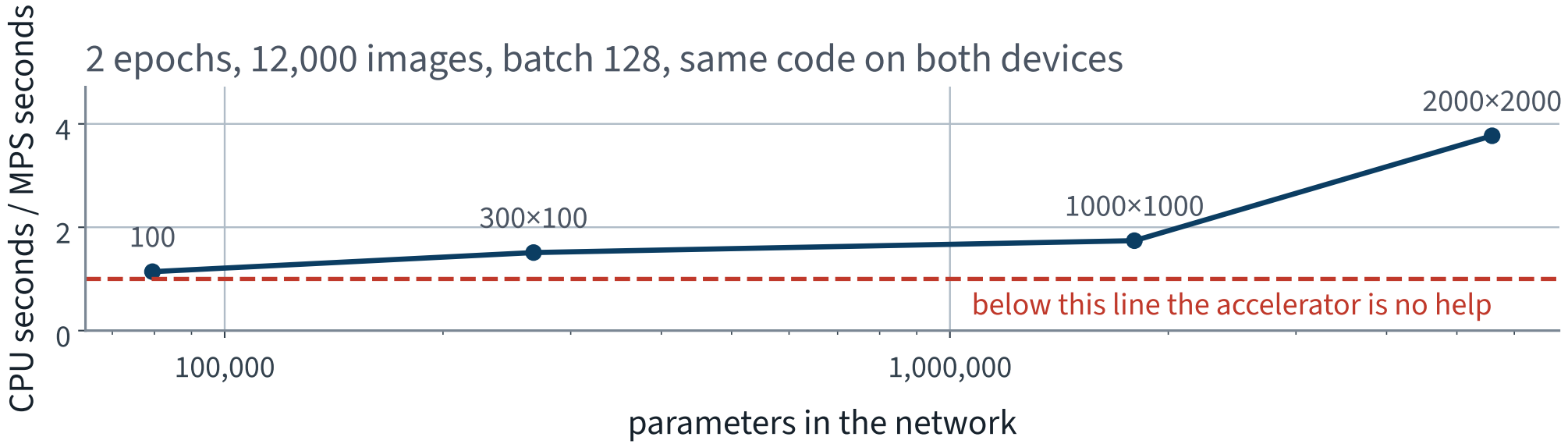
The accuracy is not the bug

`MLPClassifier`, refitted on this lecture's 12,000 images: 87.38% validation accuracy against a 10.0% baseline is a real result.

Nothing here was measured dishonestly. The split was clean, the validation set was separate, the test set was touched once.

✗ The bug is that you cannot do anything else.

The accelerator’s lead grows with the arithmetic per batch



At the smallest width the two devices nearly tie. From there the ratio climbs, because each batch carries more arithmetic to cover the cost of dispatching it.

The diagnosis

> **ONE SENTENCE**
>
> `fit()` contains a loop over epochs and batches that you did not write, cannot see, and cannot enter.

Every wall is the same wall. The repair is therefore not a better library — it is **owning the loop**.

Note what is *not* wrong: the maths, the metric, the split, the architecture. All of that carries over unchanged.

Tensors

An array that also knows two things a NumPy array does not.

```
1  import torch
2
3  a = torch.tensor([[1., 2.], [3., 4.]])
4  print(a.device)                # cpu
5  b = a.to("cuda")               # or "mps"
6  print(a.requires_grad)         # False — nothing is recorded yet
7  print(a.numpy())               # shares memory with a, on CPU
```

- **Where it lives** — an operation between tensors on different devices is an error, and a loud one
- **Whether to record** — the subject of `requires_grad`, two slides on

Finding the device, portably

```
if torch.cuda.is_available():
    device = "cuda"                          # NVIDIA, and Colab
elif torch.backends.mps.is_available():
    device = "mps"                           # Apple Silicon
else:
    device = "cpu"                           # correct, and slow
```

NOT EXAMINABLE

Device selection is engineering, not machine learning. It is on a slide because from here to the end of the course, getting it wrong is the difference between a cell you wait for and one you walk away from.

Why `requires_grad` builds a graph

It does not compute a derivative. It says *record what happens to me*.

- Every operation on a recording tensor creates a node holding the operation and its inputs
- The graph is built **during the forward pass**, by running it — there is no separate compilation step
- It is rebuilt from scratch on every batch, which is why a Python `if` inside `forward` just works

That is the forward sweep from this lecture's mathematics block, with the graph written down as it goes so the reverse sweep has something to walk.

You can look at the graph

```
>>> u = torch.tensor(2.0, requires_grad=True)
>>> v = torch.tensor(3.0, requires_grad=True)
>>> q = u * v + torch.sin(u)
>>> r = q ** 2
>>> r.grad_fn
<PowBackward0 object at 0x1291a4b50>
>>> r.grad_fn.next_functions
((<AddBackward0 object at 0x1291a4be0>, 0),)
```

Those are the nodes in the diagram, by name. Walking `next_functions` to the leaves *is* the reverse sweep.

And you can turn it off

```
with torch.no_grad():
    preds = model(X_val)          # no graph is built
```

- Evaluation needs no derivatives, so recording them wastes memory proportional to the size of the batch
- On a large validation set the difference is the difference between fitting on the GPU and not

✗ Forgetting `no_grad` is slow and memory-hungry, but it is *not* a correctness bug. The two bugs that follow are.

The training loop

```
model = build_model().to(device)
opt   = torch.optim.Adam(model.parameters(), lr=1e-3)
lossf = nn.CrossEntropyLoss()

for epoch in range(EPOCHS):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()                    # 1
        out  = model(xb)                   # 2
        loss = lossf(out, yb)              # 3
        loss.backward()                    # 4
        opt.step()                         # 5
```

Five lines. There is no hidden machinery: this is the whole of what `fit()` was doing.

One turn per mini-batch



Every arrow is a line you wrote, so every arrow is a place you can print, clip, log, branch or stop.

Line by line

Line	What it does
`model.train()`	puts dropout and batch-norm layers into training behaviour
✗ `opt.zero_grad()`	✗ clears `p.grad`, which `backward()` *adds* to
`model(xb)`	the forward sweep; the graph is recorded as it runs
`lossf(out, yb)`	reduces the batch to one scalar — the whole point of the reverse sweep
`loss.backward()`	the reverse sweep; fills `p.grad` for every parameter
`opt.step()`	updates the parameters from `p.grad`

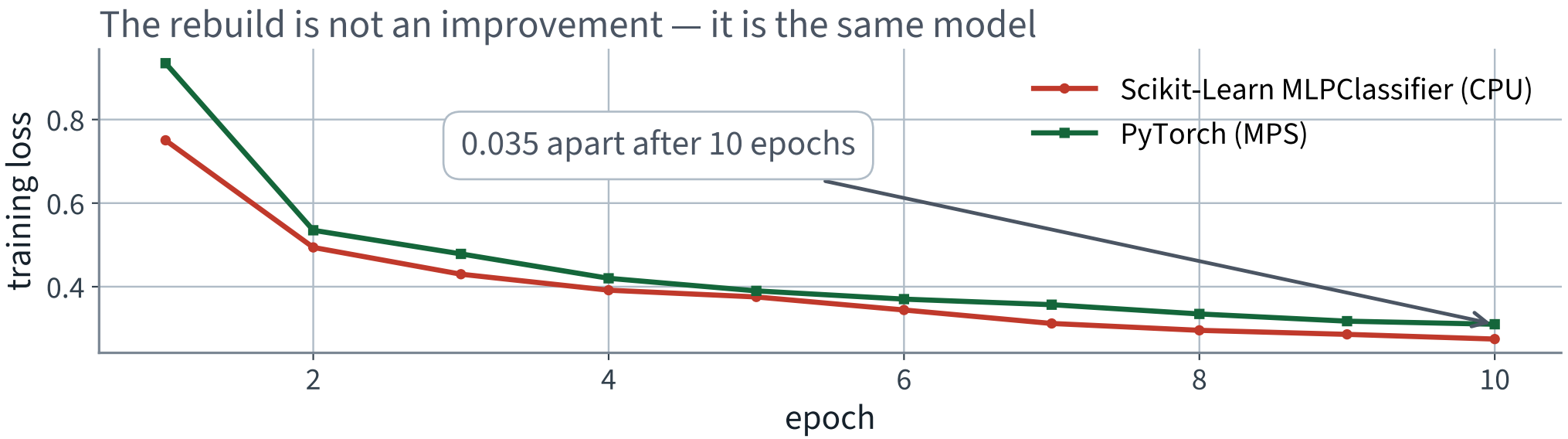
Run it

Model, 10 epochs on 12,000 images	Validation accuracy
Scikit-Learn, CPU	87.38%
✓ PyTorch, MPS	**86.32% ✓**

The accuracies agree, as they must.

Same architecture, same optimiser, same learning rate, same epochs, same data. **The rebuild is not an improvement — it is the same model.** What we bought is the loop.

Two libraries, one model



If these curves had disagreed, one of the two implementations would be wrong. Overlaying them is the first check after any rewrite.

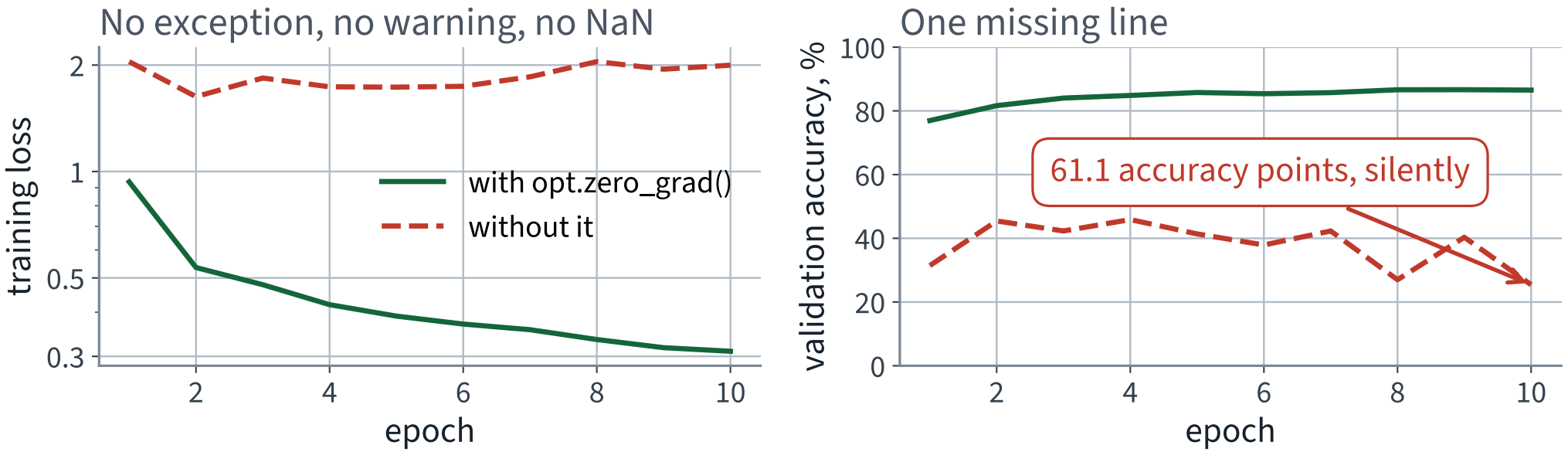
The first bug that never raises

```
for xb, yb in train_loader:
    out  = model(xb)
    loss = lossf(out, yb)
    loss.backward()
    opt.step()
```

One line has been removed. It runs. The loss goes down.

Take thirty seconds before the next slide.

One missing line, measured



✗ **Both runs completed without error. One of them is a working classifier and the other is not.**

The cost of the missing `zero_grad()`

	Final training loss	Validation accuracy
With `opt.zero_grad()`	0.3082	**86.32% ✓**
✗ Without it	**✗ 2.2441**	**✗ 20.44%**

65.88 points

The missing line is `opt.zero_grad()`. `backward()` *adds* to `p.grad`, so every step uses the sum of all the gradients so far. 10 epochs on 12,000 images, identical seeds, identical everything else — and no exception, no warning, no `NaN`.

The second bug that never raises

```
model = build_model(dropout=0.2)
train(model, ...)

with torch.no_grad():
    acc = (model(X_test).argmax(1) == y_test).float().mean()
print(f"test accuracy {acc:.4f}")
```

Correct output type, plausible value, no error.

Same question. Thirty seconds.

`train()` and `eval()`

Some layers behave differently in the two phases. Two of them matter in this course:

Layer	In `train()`	In `eval()`
Dropout	zeroes a random fraction of units	passes everything through
Batch normalisation	uses this batch's statistics	uses the running averages

✗ The module defaults to training mode. Build a model, never call `eval()`, and every evaluation you make is of a randomly crippled network.

Batch normalisation arrives in Lecture 9; the same failure applies to it and is harder to see, because it does not fluctuate.

> **FAILURE CONDITION — THE MODULE IS STILL IN TRAINING MODE**
>
> `nn.Module` defaults to training mode, so dropout stays active and batch normalisation keeps updating its running statistics through every evaluation. Nothing raises. The number is simply of a different model than the one you meant to score.

The cost of the missing `model.eval()`

Evaluation mode	Test accuracy
✓ `model.eval()`	**85.54% ✓**
✗ `model.train()`, **mean of 10 calls**	**✗ 84.75%**
lowest of those 10	84.53%
highest of those 10	84.97%

0.79 points

Dropout 0.2, measured on 10,000 test images. Look at the spread: a deterministic function of fixed weights and fixed data returns the same number every call, so a metric that wobbles between calls has a layer still in training mode.

What the two bugs have in common

	Missing `zero_grad()`	Missing `eval()`
Raises?	no	no
Warns?	no	no
Produces a plausible number?	✗ yes	✗ yes
Cost, measured today	✗ 65.88 points	✗ 0.79 points
Caught by	reading the loop	running it twice

Both are one line. Both survive a code review that is looking for exceptions rather than for behaviour.

`Dataset` and `DataLoader`

```
from torch.utils.data import TensorDataset, DataLoader

train_dl = DataLoader(TensorDataset(X_fit, y_fit),
                      batch_size=128, shuffle=True,
                      generator=torch.Generator().manual_seed(42))
```

- `Dataset` answers two questions: how many, and give me number i
- `DataLoader` shuffles, batches, and can load in parallel while the GPU is busy

Indexing a big tensor by hand works while the data fits in memory. It stops working once the data does not, and the interface does not change.

Averaging the metric per batch, measured

10,000 test images in batches of 384 give 27 batches, the last containing 16. A plain mean of per-batch accuracies weights that short batch exactly as heavily as a full one.

How the accuracy was aggregated	Value
✓ Counted over the whole set	85.350% ✓
✗ Mean of the per-batch accuracies	✗ 85.204%

0.15 accuracy points, and the per-batch mean is the *lower* one: those 16 went worse than the rest. Small *here*, because the short batch is 16 of 384. With batch size 3,000 on 10,000 images it would carry a quarter of the weight instead of a tenth.

FAILURE CONDITION — A METRIC AVERAGED PER BATCH

Weight by the batch size, or count over the set. The error is not always large; the point is that **you cannot tell from the number whether it was.**

A custom module

```python
class Sorter(nn.Module):
    def __init__(self, widths=(300, 100), n_in=784, n_out=10):
        super().__init__()
        sizes = (n_in,) + tuple(widths)
        self.blocks = nn.ModuleList(
            nn.Linear(a, b) for a, b in zip(sizes, sizes[1:]))
        self.head = nn.Linear(sizes[-1], n_out)

    def forward(self, x):
        for blk in self.blocks:
            x = torch.relu(blk(x))
        return self.head(x)
```

`super().__init__()` before anything else: it creates the registry the layers are recorded in, and assigning a layer before it raises `AttributeError: cannot assign module before Module.__init__() call`. Everything after that — `.to(device)`, `.parameters()`, `state_dict()` — comes for free.

Searching the hyperparameters properly

The previous lecture tuned by hand: ten points, one axis at a time. **Optuna** proposes a configuration, trains it briefly, scores it, and lets a sampler choose the next from what it has learned.

```
def objective(trial):
    lr   = trial.suggest_float("lr", 1e-4, 1e-2, log=True)
    drop = trial.suggest_float("dropout", 0.0, 0.5, step=0.1)
    return validation_accuracy(train(lr=lr, dropout=drop, epochs=4))

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=8)
```

> **NOT EXAMINABLE**
>
> Optuna is not in the book. It is here because it is what you will actually use, and because it makes the point that hand tuning was never a search.

Saving the model

```
torch.save(model.state_dict(), "sorter.pt")          # the tensors

fresh = build_model().to(device)
fresh.load_state_dict(torch.load("sorter.pt", weights_only=True))
```

The tensors are 1041 KB — 266,610 float32 numbers, and this is their size, not the file's. The notebook prints what the file actually weighs, and it is a little more: `state_dict` is written as a zip archive, so the names and the container are in there too.

> **WHAT NOT TO DO**
>
> `torch.save(model, ...)` pickles the object, and unpickling it needs your class definition at the same import path. Next term, when the file has moved, the checkpoint will not load.

And check the reload

```
assert accuracy(model, X_test, y_test) == accuracy(fresh, X_test, y_test)
```

`accuracy` counts hits over the whole set, in `eval()` mode, under `no_grad`. Not “it looks about the same”: a reloaded network is either the same function or it is not, and the difference is one assertion.

The usual cause of failure is loading a `state_dict` into a model built with different widths. PyTorch raises for that — but only if you did not pass `strict=False` to silence it.

Where we ended

Model	Validation accuracy
Always one class	10.0%
Scikit-Learn MLP, CPU	87.38%
✓ PyTorch, same model, MPS	**86.32% ✓**
✗ The same code without `zero_grad()`	**✗ 20.44%**

The PyTorch model's test accuracy, touched once: 85.35%. The last row is the one to remember. It ran, it finished, and it reported a number.

The silent-failure catalogue so far

Failure	Diagnosed in	Measured cost
Scaler fitted before the split	Part I	nothing measurable
Evaluating on training data	Part I	an RMSE of zero
Accuracy under class imbalance	Part I	a useless model above 90%
Hyperparameters chosen on the test set	Part I	an optimistic report
✗ Missing `optimizer.zero_grad()`	**✗ today**	**✗ 65.88 points**
✗ Missing `model.eval()`	**✗ today**	**✗ 0.79 points**
✗ Metric averaged per batch	**✗ today**	**✗ 0.15 points**

Every one of them ran to completion and printed a plausible number.

The notebook for this lecture

Open Lecture 8 in Colab

- **Runs on free CPU.** The corpus is subsampled to 12,000 images, and the numbers on these slides are its run
- Autograd checked against the hand derivative, then the wrapper timed against the loop written out line by line
- All three silent bugs, reproduced and measured — a missing `zero_grad()`, a missing `model.eval()`, a metric averaged per batch
- `DataLoader`, a custom `Module`, an Optuna search, and a checkpoint that reloads to the same function

> **WHAT TO CHANGE**
>
> Make the network four times deeper and re-run the training loop. It trains far worse than its size suggests it should — and the next lecture derives why, and repairs it.

Design exercise · in the manner of an ML system-design interview

The task: the training pipeline behind a ride-hailing app’s arrival-time model

> **THE BRIEF**
>
> A ride-hailing app shows every rider, at booking, how many minutes away the driver is — a network fitted on past trips, which several engineers **retrain every week**. Too early and the rider cancels; too late and the booking is lost. The pipeline takes a configuration and produces a checkpoint, a validation error and a record of both, so a colleague can reproduce the number and a reviewer release it.

- **Given:** the trip log — request, pickup, driver position at dispatch, traffic, arrival — the model class, and the loop written out
- **Scale:** millions of trips a week; a model that fits on one machine; many runs a day
- **Consumers of the output:** the engineer who retrains, the reviewer who releases, the service behind every booking

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today’s: the five-line loop, owned, and three bugs that never raise.

Step 1 — Frame it: a number nobody can reproduce is not a release

- A run outputs three things: the checkpoint, the mean absolute error in minutes *counted over the whole validation set*, and the configuration and seed behind them
- The baselines: the rule the app shipped first — distance at the city's average speed — and last week's checkpoint, scored the same way on the same split
- The reference: the same network under the convenience wrapper, whose loss curve the loop must overlay

> **PITFALL**
>
> Every failure in this lecture ran to completion and printed a plausible number; a gate that looks for exceptions passes all of them. Judge runs on behaviour: the same configuration twice gives the same curve, the loop's sits on the wrapper's, and the new checkpoint beats last week's on the same split, or it is not released.

Step 2 — `Dataset`, `DataLoader`, and a split written to disk

- A `Dataset` answers how many, and give me trip i; a `DataLoader` shuffles and batches, and the interface does not change when a week of trips stops fitting in memory
- Split once per retrain, by time — the latest days are validation — and save the indices; every engineer's validation set is the same file
- Shuffle the training loader with a seeded `Generator`; the validation loader is not shuffled at all

> **PITFALL**
>
> `shuffle=True` with no generator: every run sees the batches in a different order, and two engineers comparing a tenth of a minute are comparing shuffles. A split redone from a different seed on another machine moves trips between training and validation, and the number is of a different experiment. One split file, one generator, both named in the configuration.

Step 3 — The loop, written out, and the first bug that never raises

- `model.train()`, then per batch: `zero_grad`, forward, loss, `backward`, `step` — five lines, every one a place to print, clip, log or stop
- The loss is the one the product asked for, the absolute error in minutes, which the wrapper had no argument for; one loop function for the team, reviewed once, the per-epoch loss logged

> **PITFALL — THE MISSING `zero_grad()`**
>
> `backward()` *adds* to `p.grad`, so without the clearing line every step uses the sum of all gradients so far. It runs, the loss falls for a while, and the model is little better than the rule it replaced, and nothing raises. It is caught by reading the loop — which is why the team has one — and by the overlay against the wrapper's curve.

Step 4 — One `evaluate()` for everyone: `eval()`, `no_grad`, counted

Rule	Without it
`model.eval()` first	dropout stays on and the error is of a randomly crippled network
under `torch.no_grad()`	a graph is recorded for nothing: slow and memory-hungry, though not wrong
count over the whole set	a plain mean of per-batch errors weights the short last batch as a full one
run it twice	an error that wobbles between identical calls has a layer still in training mode

PITFALL

Each engineer writes their own evaluation and each omits a different line. The missing `eval()` cost under a point today and the per-batch mean a small fraction of one; neither shows in the number, and either decides a release. One function, called by the loop, the search, the reload check and the gate.

Step 5 — Seeds: the same run twice, then several runs before a claim

- Seed everything that draws a random number — the weights, the loader's generator, the split — and record the seed with the configuration; the same seed on the same machine must give the same curve
- A difference between two configurations is a claim only when it is larger than the spread between seeds of one configuration: run a few seeds and report mean and spread

> **PITFALL**
>
> An engineer reports a gain of a few seconds of error from a new dropout rate, measured on one seed against a colleague's one seed. The seeds alone move the number by as much, and the release meeting argues about a change that did nothing. Reproduce first, repeat second, and only then compare.

Step 6 — A hyperparameter search with a budget

- Hand tuning was never a search. An Optuna `objective` proposes a configuration, trains it for a few epochs, scores it with the team's `evaluate()` on validation, and the sampler chooses the next
- Fix the number of trials and the epochs per trial in advance, so the search fits the week's retrain, and log every trial, failed ones too
- Retrain the winner with the full budget, then read the test split — the last day, held back — once

> **PITFALL**
>
> The best trial's validation error is reported as the model's. It is the minimum of many draws on the set it was chosen on, so it is optimistic; and it was earned on a short budget, so the ranking is a measurement of that budget. The search chooses a configuration, not a number: the number comes from the retrained winner, on trips it never saw.

Step 7 — Save the tensors, rebuild the model, and check the reload

- `torch.save(model.state_dict(), path)` — the tensors and their names, nothing else — beside the configuration that built it, so the service can call `build_model(config)` without the training code
- Reload with `weights_only=True` into a fresh model and assert that `evaluate()` returns *exactly* the same error — in the pipeline, and in the service at load

> **PITFALL**
>
> `torch.save(model, path)` pickles the object, and unpickling needs the class at the same import path — the service has another, and the checkpoint fails at release. `strict=False` loads a `state_dict` into a model of different widths in silence, layers missing, and every rider gets an estimate nobody validated. A reloaded network is either the same function or it is not: one assertion.

Step 8 — In production: the weekly release, the rollout, and the clock

- A checkpoint that passes the gate goes to a share of bookings in one city against last week's, read on error against real arrivals and on cancellations — the A/B test; last week's file is the rollback, its engineer on call
- Wall-clock is a measurement of one machine: time pairs of runs that differ in one thing — framework on one CPU, device with one code — and record machine, device and batch size

> **PITFALL**
>
> "PyTorch on the GPU is several times faster" — one number for two changes, read on one laptop, in a hardware ticket. Taken one at a time, the device alone nearly ties at the smallest width. A second is not a figure: it is a reading from a named machine, stored as one.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Configuration and seed → the split file by time → seeded loaders → the one loop, overlaid on the wrapper → the one `evaluate()`, run twice → seeds before a claim → a budgeted search, the winner retrained → `state_dict` with configuration, the reload asserted → timings, machine named → one city, the A/B test, rollback ready.

Pitfall	Address it by
A plausible number from a silent bug	behaviour: the overlay, the same curve twice, last week's beaten
Shuffles and splits that differ per machine	one split file, one seeded generator, both in the configuration
A missing `zero_grad()`	one reviewed loop for the team
A missing `eval()`, a per-batch mean	one `evaluate()`, counted, run twice
A gain measured on one seed	mean and spread over a few seeds
The best trial reported as the result	the winner retrained on the full budget; the test set once
A pickled model, a silent `strict=False`	`state_dict` plus configuration; an exact reload assertion
One number for two changes	paired timings, one change each, the machine named

What we did

1. Derived reverse-mode differentiation: the chain rule is a product of Jacobians, and bracketed from the output end it costs one sweep per output
2. Concluded that training — many parameters, one scalar loss — leaves reverse mode as the only viable choice; that is what backpropagation is
3. Found that the wrapper costs control, not accuracy, and that an accelerator's lead grows with the arithmetic per batch
4. Wrote the training loop — tensors, `requires_grad`, the graph, `backward()`, the optimiser step — and got the same model back
5. Met three bugs that raise nothing: a missing `zero_grad()`, a missing `model.eval()`, a metric averaged per batch
6. Added `DataLoader`, a custom `Module`, an Optuna search, and a checkpoint checked by assertion
7. Designed the training pipeline behind a ride-hailing app's arrival-time model in nine steps — runs judged on behaviour, one split file by time and seeded loaders, one reviewed loop, one `evaluate()` run twice, seeds repeated before a claim, a budgeted search with the winner retrained, a checkpoint asserted on reload in the service too, and a weekly release with an A/B test and a rollback — each step named for the pitfall it avoids

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 8

five, in the style of the written paper

Exercises — Lecture 8

1 Reverse-mode automatic differentiation costs one sweep for all partial derivatives; forward mode costs one per input. State the condition on the function's shape that makes reverse mode the right choice. [4]

2 Test accuracy on 10,000 images is computed as the plain mean of the per-batch accuracies, in batches of 384. State why this is wrong, and the rule that fixes it. [3]

3 The five lines of the training loop are reordered so that `opt.step()` comes before `loss.backward()`. State what the student observes. [4]

Solutions on Lecture 9's deck.

Exercises — Lecture 8 (continued)

4 A network with dropout is evaluated without calling `model.eval()`. Describe the symptom precisely, and the one-line diagnostic that catches it. [5]

5 In the measured comparison the accelerator barely beat the CPU on the smallest network, and its lead grew as the layers widened. Give the reason, and the quantity that decides where the crossover sits. [4]

Solutions on Lecture 9’s deck.

THE FIVE SET AT THE END OF LECTURE 7

Solutions Lecture 7

not part of this lecture

Solutions — Lecture 7

1. A multilayer perceptron with the activation removed is written as $W_3 W_2 W_1 x$. State what function class it…

✓ Only linear maps — the product of matrices is a matrix.

- Composition of linear maps is linear, whatever the depth
- The nonlinearity is what makes an extra layer a new function rather than a re-parameterisation

2. Give the parameter count of a dense layer from m inputs to n units, and evaluate it for the first layer…

✓ $mn + n$; here $784 \times 300 + 300 = 235{,}500$.

- One weight per input-output pair, plus one bias per unit
- The first layer of an image network usually dominates the count, because m is the pixel count

Solutions — Lecture 7 (2)

3. A ten-class classifier for mutually exclusive classes is built with ten sigmoid output units and binary…

✓ It is the multilabel row. Mutually exclusive classes are the multiclass row: ten units, softmax, cross-entropy.

- Ten sigmoids ask ten independent yes/no questions, and their outputs need not sum to 1
- Softmax couples the outputs into one ten-way question, and cross-entropy with a one-hot target is minus the log probability of the true class

4. Fashion-MNIST is exactly balanced across ten classes. State the trivial baseline's accuracy, and why…

✓ 10%. With equal class sizes and equal costs, accuracy is not dominated by one class.

- The majority-class predictor scores the largest class share, which is one tenth
- Lecture 3's objection to accuracy was imbalance, and it does not apply

Solutions — Lecture 7 (3)

5. An architecture sweep finds the best hidden-layer size on this dataset. State what that result does and does…

✓ Very little. It holds for this task at the training-set size you swept at. In the lecture the order survived a move from 6,000 to 12,000 images, but that is a measurement on this corpus, not a guarantee: only a re-run on all the data says whether a subset's ranking was real.

- A deeper network needs more data to pay for its extra parameters, so a ranking measured on a fraction of the data can change on the rest
- It is a measurement of one sweep at one size, not a rule. Reporting it as a general recommendation is the error

LECTURE 9 · GÉRON CH. 11

Training deep networks

A twenty-layer network that will not train, instrumented until it says why
The variance recursion, and the Glorot and He initialisations it gives
Six repairs — initialisation, activation, normalisation, clipping, optimisers, schedules — and an ablation of which one paid

Run the Lecture 8 notebook first